Extracting Q Values from Flow Matching Imitation Policies for Online Robot Learning

Jialong Ning Harshit Moondra Zhaopeng Wang
Karthik Seelam Sreenivas
CS 59300 Midterm Report

Abstract

Flow-matching policies model robot actions from demonstrations, but applying Q2RL-style value extraction requires likelihood and entropy estimates beyond their standard sampling interface. In this work, we study value estimation from a frozen flow-matching imitation policy. We represent arm commands with a conditional flow and gripper commands with a categorical head, and compute their joint log probability through continuous change of variables. Combining this probability with policy entropy and a state-value model yields an imitation action score for Q-gating against a separate reinforcement-learning policy. As a second route to online improvement that also keeps the imitation policy frozen, we implement a DSRL-style learner that steers the flow through its input noise. On Robomimic Lift, Can, and Square, frozen Flow BC succeeds in 60%, 60%, and 15% of evaluation episodes. In single-seed full-budget runs, latent steering raises success from 46% to 92% on Lift and from 52% to 88% on Can, measured against BC on the same 50 evaluation episodes. On Square, where only 9 of 100 warmup episodes succeed, its critic diverges and success falls to zero. The planned evaluation examines action-value calibration, online sample efficiency, and computation cost, with a shared-critic gate isolating the effect of the proposed scoring rule.

1 Introduction and research question

Learning manipulation from human demonstrations provides a practical starting point for robot control. Behavior cloning (BC) fits a conditional action distribution to demonstrated behavior, and flow matching offers an expressive representation of this distribution through a learned velocity field [3]. Once deployed, however, a BC policy continues to sample from the same learned distribution. Task rewards become useful for improvement only when an additional learning mechanism connects them to the policy's decisions.

One approach is to retain the imitation policy and train a separate reinforcement-learning (RL) policy that proposes alternative actions. This creates an action-selection problem: at each state, the system must decide whether to execute the imitation proposal or the RL proposal. Q2RL addresses this problem by constructing an imitation value estimate from policy likelihood, entropy, and state value [1]. Applying this construction to a flow policy requires evaluating quantities that are absent from its usual sampling procedure. It also requires examining whether the resulting score agrees with task return closely enough to guide action selection.

In this work, we formulate online improvement of a flow-matching imitation policy as value estimation followed by Q-gating. The imitation model remains frozen. We use its continuous-flow representation to evaluate action log probability, estimate entropy by sampling, and combine these quantities with a state-value model fitted to imitation rollouts. A separate RL actor and critic learn from executed transitions. At each control step, the gate compares the imitation score with the RL critic's score and executes the higher-valued proposal. Our research question is:

Can value estimates derived from a frozen flow-matching imitation policy improve imitation/RL action selection and online sample efficiency over a shared-critic gate under matched interaction budgets?

Relative to the extended abstract, we separate the quality of value estimation from its effect on online control, and focus the initial study on low-dimensional, single-step manipulation. This makes the density calculation tractable and allows a controlled comparison of scoring rules. We hypothesize that a fixed imitation score can support

demonstrated behavior early in training while the RL policy learns useful alternatives. Because the gate needs both a calibrated imitation score and a competent RL policy, we also evaluate latent-noise steering [8], which adapts the same frozen flow without a density query and provides an independent reference for how much online improvement the frozen policy allows.

Our contributions at this stage are:

1. A hybrid flow imitation policy with joint likelihood and entropy estimation, coupled to a rollout-based state-value model for scoring imitation actions.
2. A matched Q-gating implementation and evaluation protocol that isolate the scoring rule through shared-critic comparisons, return calibration, and computational ablations.
3. A latent-steering learner for the frozen hybrid flow, a GPU training setup for it, and a staged pilot on Can with paired comparisons against BC (Section 5.3).

2 Related work

Imitation-guided online learning. IBRL uses a pretrained imitation policy to supply action proposals for online exploration and target-value bootstrapping [2]. Q2RL further constructs an imitation action-value estimate from an explicit BC distribution and compares it with a separately learned RL value [1]. These approaches establish how an imitation policy can guide subsequent interaction, but the likelihood and entropy required by Q2RL are not directly available from a standard flow sampler. We extend this interface to a hybrid flow policy and compare its score with a shared online critic. The shared-critic condition is inspired by IBRL; its initialization and target updates follow our implementation.

Flow matching and continuous density estimation. Flow Matching learns a velocity field that transports samples from a simple prior toward the data distribution [3]. FFJORD evaluates continuous-flow densities through a Jacobian trace and uses Hutchinson estimation to reduce the cost of that calculation [4]. These methods provide the training and density-estimation tools used here. Our task additionally requires relating a conditional action density to manipulation return. We therefore examine both numerical accuracy and the calibration of the resulting value score.

Reinforcement learning for flow policies. ReinFlow introduces learnable noise into the generation process to support online policy-gradient updates [5]. FlowDPG distills critic-gradient corrections into the flow's velocity field [6], and Q-VGM constructs critic-guided velocity targets for flow-based vision-language-action policies [7]. These methods update the action generator itself. Our formulation keeps the imitation generator fixed and trains a separate RL policy, with adaptation occurring through the selection of their proposals.

Latent steering of frozen policies. DSRL adapts a frozen diffusion or flow policy by learning its latent-noise input distribution [8]. Its action-to-noise critic distillation allows learning from transitions labeled with executed actions. Both DSRL and our gating approach preserve the base generator, but they act at different points in the control pipeline: DSRL changes the generator's input distribution, while our gate selects between an imitation action and an independently learned RL action. We implement a DSRL-style learner for our hybrid flow (Section 3.6) and use it as a second online-improvement method on the same frozen policies. We do not copy upstream code, and we do not claim an exact reproduction of DSRL.

3 Method

3.1 Problem formulation and overview

We consider a finite-horizon manipulation MDP with physical observation s_k, executed action a_k, reward r_k, horizon H, and discount $\gamma = 0.99$. The goal is to maximize

$$J(\pi) = \mathbb{E}_{\pi}\left[\sum_{k=0}^{T-1} \gamma^k r_k\right], \qquad T \le H\,. \tag{1}$$

The simulator provides a sparse success reward, transformed as $r_k = 5r_k^{\text{raw}} - 1$. Success, simulator termination, or the horizon ends an episode. The value and RL networks receive $x_k = (\bar{s}_k, 1 - k\,/\,H)$, where $\bar{s}_k$ is the

normalized physical observation. The imitation policy uses the physical observation alone. This distinction lets the value models represent the finite-horizon return without changing the BC input.

Given demonstration pairs $\mathcal{D} = \{(s_i, a_i)\}_{i=1}^{N}$, we first learn a conditional flow policy π_{FM}. Complete rollouts of this policy then supply return targets for a state-value model V_ψ. We freeze both models and use their imitation score to guide online interaction with a separate actor π_{RL} and twin critics Q_{ϕ_i}. The same demonstration split, frozen policy, and initialization are used in both gating conditions.

3.2 Flow matching imitation policy

Each action has six continuous arm commands and one gripper command, $a = (b, g)$, with $b \in (-1, 1)^6$ and $g \in \{-1, 0, 1\}$. A conditional flow models the arm commands, while a categorical head $p_\eta(g \mid s)$ models the gripper. Demonstration gripper commands are mapped to their signs, matching the controller's discretization. The two heads are conditionally independent given state.

For a demonstrated arm command b, we define $u_1 = \operatorname{atanh}((1-\epsilon)b)$ with $\epsilon = 0.001$, sample $u_0 \sim \mathcal{N}(0, I_6)$ and $t \sim \mathcal{U}[0,1]$, and construct $u_t = (1-t)u_0 + tu_1$. The target transport velocity is $u_1 - u_0$. The BC loss is

$$\mathcal{L}_{\mathrm{BC}} = \mathbb{E}\left[\frac{1}{6}\left\|v_\theta(u_t, t \mid s) - (u_1 - u_0)\right\|_2^2\right] - \mathbb{E}\left[\log p_\eta(g \mid s)\right]. \tag{2}$$

The first term learns the arm velocity field, averaging squared error over six coordinates. The second term fits gripper probabilities. Inward smoothing makes boundary demonstration commands finite after the inverse tanh transform. It is applied during training only.

At inference, we sample u_0 from the Gaussian prior and integrate $du_t / dt = v_\theta(u_t, t \mid s)$ from $t = 0$ to $t = 1$. The arm action is $b = \tanh(u_1)$, and the gripper is sampled from p_η. Here t is generative flow time; k indexes environment control steps. The velocity network has two 128-unit hidden layers with SiLU activations and bounded output $6\tanh(v_{\mathrm{raw}} / 6)$. We use 16 RK4 steps and apply no clipping to sampled arm commands. The learned policy and normalization statistics remain fixed during online learning. We write $f(s, z)$ for the deterministic map from base noise $z = u_0$ to the arm command; Section 3.6 steers the policy through this map.

3.3 Likelihood and entropy estimation

The flow defines a sampling map from Gaussian noise to arm commands. To evaluate a command b, we invert the tanh transform and integrate the flow backward to recover its base point numerically. Continuous change of variables gives the joint log probability

$$\log \pi_{\mathrm{FM}}(b, g \mid s) = \log p_0(u_0) - \int_0^1 \nabla_u \cdot v_\theta(u_t, t \mid s)\, dt - \sum_{j=1}^{6} \log(1 - b_j^2) + \log p_\eta(g \mid s). \tag{3}$$

The integral accounts for volume change under the flow, the summation is the tanh Jacobian correction, and the final term is the gripper probability mass. This joint distribution uses Lebesgue measure for the arm and counting measure for the gripper. Density queries require arm commands strictly inside $(-1, 1)^6$.

We evaluate the divergence using either the exact Jacobian trace or a Hutchinson approximation, with each random probe held fixed along an integration path. Both use numerical ODE integration, so their accuracy also depends on the integration step count. Entropy is estimated from independent policy samples:

$$\hat{\mathcal{H}}(s) = -\frac{1}{M}\sum_{m=1}^{M} \log \pi_{\mathrm{FM}}(a^{(m)} \mid s), \qquad a^{(m)} \sim \pi_{\mathrm{FM}}(\cdot \mid s). \tag{4}$$

The default is $M = 8$. We compute the Monte Carlo standard error and use natural logarithms throughout, so the entropy and likelihood terms share the same units.

3.4 Value estimation from the frozen policy

The state-value model supplies the return scale for the imitation score. For each complete BC rollout, we compute $G_k = \sum_{j=k}^{T-1} \gamma^{j-k} r_j$ and fit $V_\psi(x_k)$ by squared error to these Monte Carlo targets. The value model is then frozen. Following the Q2RL construction, we define

$$\tilde{Q}_{\mathrm{FM}}(x, a) = V_\psi(x) + \alpha_Q \Big[\log \pi_{\mathrm{FM}}(a \mid s) + \hat{\mathcal{H}}(s) \Big]. \tag{5}$$

For exact likelihood and entropy, the bracketed term has zero expectation under π_{FM}. Interpreting it as a task advantage requires the imitation distribution to be approximately Boltzmann with respect to the task Q-function. Equation (5) is therefore an approximate value model whose return calibration and action ranking will be evaluated directly.

Before online interaction, we initialize the RL actor by likelihood fitting to the collected BC actions and initialize its critics against the frozen scores. The initial trajectories also seed replay. Both gating conditions load these same artifacts, holding value-based initialization fixed while changing how the imitation proposal is scored online. Warmup interactions are included in the training budget.

3.5 Online reinforcement learning and Q gating

The RL actor uses a tanh-Gaussian arm head and a categorical gripper head. Twin critics learn discounted task returns with the target

$$y = r + \gamma(1-d) \min_i Q_{\bar{\phi}_i}(x', a'), \qquad a' \sim \pi_{\mathrm{RL}}(\cdot \mid x'), \tag{6}$$

where d indicates episode termination and $\bar{\phi}_i$ denotes target-network parameters. The actor minimizes $\mathbb{E}\Big[\alpha_{\mathrm{SAC}} \log \pi_{\mathrm{RL}}(a \mid x) - \min_i Q_{\phi_i}(x, a)\Big]$, with exact enumeration of the three gripper choices. Critic targets exclude entropy and remain in task-return units. The imitation-score coefficient α_Q and actor entropy coefficient α_{SAC} are separate, with defaults 1 and 0.01, respectively.

At each control step, the frozen flow samples a_{FM}. The RL actor supplies a_{RL} by applying tanh to its Gaussian arm mean and choosing its most likely gripper command. With $\underline{Q}_{\mathrm{RL}} = \min_i Q_{\phi_i}$, Q-gating executes

$$a_k = \begin{cases} a_{\mathrm{RL}}, & \underline{Q}_{\mathrm{RL}}(x_k, a_{\mathrm{RL}}) > \tilde{Q}_{\mathrm{FM}}(x_k, a_{\mathrm{FM}}), \\ a_{\mathrm{FM}}, & \text{otherwise.} \end{cases} \tag{7}$$

Every executed transition enters replay, regardless of which policy proposed it. Original demonstrations are used for BC and are not inserted into online replay. The shared-critic gate replaces $\tilde{Q}_{\mathrm{FM}}(x, a_{\mathrm{FM}})$ with $\underline{Q}_{\mathrm{RL}}(x, a_{\mathrm{FM}})$, keeping the proposal and update mechanisms unchanged.

3.6 Latent steering of the frozen flow

Latent steering keeps both flow heads frozen and learns a state-conditioned distribution over the six-dimensional arm noise, $\pi_Z(z \mid x) = \mathcal{N}(\mu_\omega(x), \sigma_\omega(x)^2)$. The executed arm command is $f(s, z)$, and the gripper is still sampled from p_η. Following DSRL's noise-aliasing variant [8], learning has three parts. Twin action critics Q_i^A learn task returns from executed transitions, with the next action decoded from the steering policy and the gripper outcome averaged under p_η:

$$y = r + \gamma(1-d)\, \mathbb{E}_{g' \sim p_\eta} \Big[\min_i Q^A_{\bar{\phi}_i}(x', (f(s', z'), g')) \Big], \qquad z' \sim \pi_Z(\cdot \mid x'). \tag{8}$$

A noise critic Q^Z is then distilled at replay states from latent proposals drawn half from the prior and half from π_Z. Its regression target is $\mathbb{E}_g \Big[\min_i Q_i^A(x, (f(s, z), g)) \Big]$, with the flow and teacher detached. This lets the method

learn from transitions labeled only with executed actions, with no inverse flow or latent labels. Finally, the steering policy minimizes

$$\mathbb{E}_{z \sim \pi_Z}\Big[-\min_i Q_i^Z(x, z)\Big] + \beta\, \mathrm{KL}(\mathcal{N}(\mu_\omega, \sigma_\omega^2) \,\|\, \mathcal{N}(0, I)), \tag{9}$$

with $\mu_\omega = 2\tanh(\cdot)$ and $\sigma_\omega \le 1$. No gradient passes through the flow. The steering policy is initialized to exactly $\mathcal{N}(0, I)$, so before training it reproduces BC, and β controls how far it may move from the demonstrated behavior. Critic targets decode with 8 RK4 steps, while executed actions use 16; on 128 Lift replay states the maximum action difference between the two settings was 1.1×10^{-3}. Unlike Q-gating, this method needs no density or entropy estimate. It changes which BC behaviors are produced rather than choosing between BC and an independent policy.

4 Evaluation protocol

Tasks and demonstrations. The study uses the original proficient-human (PH) demonstrations for Robomimic Lift, Can, and Square [9]. Each task controls a Panda robot through single-step OSC_POSE commands. Observations concatenate end-effector position and quaternion, gripper joint positions, and object features. The selected `low_dim_v141.hdf5` files use the released Q2RL BC checkpoint masks, with disjoint training and validation trajectories (Table 1). Normalization statistics are fitted to training observations only, with a standard-deviation floor of 0.02.

Table 1: Demonstration splits and manipulation task settings.

Task	State dim.	Train/valid demos	Train transitions	Valid transitions	H
Lift	19	36 / 4	1,769	223	300
Can	23	180 / 20	20,883	2,324	300
Square	23	180 / 20	27,165	2,989	500

Baselines. We will compare frozen Flow BC (A), the shared-critic gate (B), and the flow-derived gate (C). Their online scoring rules are summarized in Table 2. A provides the imitation reference. B and C use the same frozen policy, warmup trajectories, actor, and critic initialization, isolating the effect of the imitation scoring rule. An ungated RL condition is planned to measure the role of gating itself and requires an additional runner mode. Latent steering (D, Section 3.6) adapts the same frozen policy by a different mechanism. Its Can pilot is reported in Section 5.3 and full-budget runs on all three tasks in Section 5.4. It will be compared with B and C once their interaction budgets are matched.

Table 2: Online conditions on the same frozen flow. All four conditions are implemented.

Condition	Imitation score	RL score	Executed action
A: Frozen Flow BC	Not used	Not used	Always imitation
B: Shared-critic gate	$Q_{\mathrm{RL}}(x, a_{\mathrm{FM}})$	$Q_{\mathrm{RL}}(x, a_{\mathrm{RL}})$	Higher score
C: Flow-derived gate	$\widetilde{Q}_{\mathrm{FM}}(x, a_{\mathrm{FM}})$	$Q_{\mathrm{RL}}(x, a_{\mathrm{RL}})$	Higher score
D: Latent steering	Not used	$Q^Z(x, z)$	$(f(s, z), g)$, $z \sim \pi_Z$

Training and evaluation settings. BC uses 10,000 Adam updates with batch size 256 and learning rate 3×10^{-4}. The online actor and critics use two 128-unit hidden layers, the same batch size and learning rate, one update per transition, and target-update rate 0.005. Prepared configurations specify 100 BC warmup episodes, followed by 20,000 value-fitting and actor/critic initialization updates for Lift and Can, or 50,000 for Square. They allow up to 500,000 training interactions, including warmup, with evaluation every 20,000 interactions. A bounded Lift pilot will determine the feasible common budget before scaling to Can and Square. We plan at least three RL seeds per fixed BC checkpoint and 50 evaluation episodes per checkpoint. Paired reset seeds and separate evaluation environments

will be used, with final held-out seeds reserved from development. The local setup uses robosuite 1.4.1 and MuJoCo 3.2.6; the Q2RL reference uses MuJoCo 3.1.6. Controlled comparisons will share the local simulator and dataset controller settings.

Latent steering uses 100 BC warmup episodes, 500 critic-only and 1,000 actor pretraining updates, then 10^5 online steps with 20 learner updates per step (2,001,500 updates in total), batch size 128, $\beta = 3$, and a 200,000-transition replay buffer. Every 10^4 online steps, BC and the steered policy are evaluated on the same 50 reset seeds with separate sampling generators. Warmup interactions count toward the training budget, and evaluation interactions are counted separately.

Compute. BC training and evaluation take about two minutes per task on a CPU. The online learners are far more expensive: our first full latent-steering attempt on CPU nodes of Purdue's Gilbreth cluster measured about 0.05 s per learner update including simulation, or about 28 hours per task. We therefore run the learner on a workstation with two NVIDIA RTX 5080 GPUs (16 GB each, CUDA 12.8, PyTorch 2.8), one process per task. Learner updates run in float64 on the GPU. Simulation and every seeded noise generator stay on the CPU, so paired evaluations are identical to a CPU run, and unit tests check that CUDA updates match CPU updates. The machine is reached remotely over a private network.

Metrics and value diagnostics. Primary metrics are task success against total training interactions, area under the success curve over a common budget, and interactions required to exceed the frozen-BC reference. Runs that do not exceed it will be identified. We will also measure early performance relative to BC, proposal-selection fractions, and decision time, separating density cost from other computation. Evaluation data remain outside replay, and evaluation interactions are counted separately. We will report variation across RL seeds separately from binomial uncertainty within each success estimate. Because every evaluation is paired by reset seed, we also count episodes solved by only one of the two policies.

To assess Eq. (5), we plan to restore full simulator snapshots for held-out states, execute a specified first action, and continue with frozen BC. Repeated continuations estimate $Q^{\pi_{\mathrm{FM}}}(x, a)$ for measuring score bias, squared error, and within-state action-ranking agreement. This extends the current evaluator's state-value checks to action values. Numerical ablations will compare 16, 32, and 64 integration steps, exact and Hutchinson traces, and entropy sample counts. Together, these experiments will examine how numerical error and return calibration affect the gate.

5 Preliminary results

We report four completed pieces of evidence: closed-loop performance of the frozen Flow BC policies, numerical checks of the flow interface, a staged pilot of latent steering on Can, and full-budget latent-steering runs on all three tasks. The Q-gating conditions (B and C) are implemented but have not yet been run, so this report contains no gating result.

5.1 Frozen Flow BC

Each task was trained once (seed 0, 10,000 updates) and evaluated with the policy frozen on 20 reset seeds (30000–30019). Lift and Can succeed in 12 of 20 episodes, and Square in 3 of 20 (Table 3). With 20 episodes, the 95% Wilson intervals are wide. On Lift, the final validation loss (0.955) is far above the training loss (0.203), consistent with overfitting its 36 training demonstrations. Can and Square show no such gap. Sampling an action costs about 3.2 ms on a CPU, so the policy is not a bottleneck for online learning. Square is the weakest task, which matches the risk noted in Section 6.

Table 3: Frozen Flow BC, one training seed, 20 evaluation episodes per task.

Task	Success	95% Wilson CI	Train / valid loss	Eval. steps	ms / step
Lift	12/20 (60%)	[0.39, 0.78]	0.203 / 0.955	3,007	3.29
Can	12/20 (60%)	[0.39, 0.78]	0.510 / 0.480	3,758	3.18
Square	3/20 (15%)	[0.05, 0.36]	0.377 / 0.457	8,971	3.22

5.2 Numerical checks of the flow interface

Value estimation and latent steering both rely on mapping between Gaussian noise and arm commands through the trained flow. For eight fixed states per task, we decoded seeded noise to an action and encoded it back. The maximum per-coordinate round-trip error falls from 2.0×10^{-4} (Lift), 6.5×10^{-5} (Can), and 9.0×10^{-5} (Square) at 16 RK4 steps to below 2×10^{-7} at 64 steps, so integration error is small at the default step count. All 17 unit tests pass on the GPU workstation, including analytic Gaussian density, entropy and convergence tests, and tests that check that CUDA updates match CPU updates. On a synthetic two-mode flow in which only one mode is rewarded, latent steering raises the rewarded-mode fraction from 0.506 under BC to 0.99, which confirms that the learner can redirect a frozen generator. This is a synthetic check, not a robotics result.

5.3 Latent-steering pilot on Can

Before committing to the full budget, we ran a staged pilot on Can using the GPU workstation. Its Can flow was retrained there with the configuration and seed of Table 3. On the pilot's evaluation seeds (40000–40019), it also reaches 12/20 BC success. Stage B collected 50 BC warmup episodes (33 successful, 8,902 transitions). Before training, the untrained steering policy reproduced the BC actions exactly, which checks that the components are connected correctly. Starting from this shared buffer, run C trained offline only (4,000 updates). Runs D and E each added 5,000 online steps at 5 updates per step (29,000 updates in total), with $\beta = 3$ (D) and $\beta \in \{1, 10\}$ (E). The four runs used both GPUs in parallel and finished in 21 minutes.

Table 4: Latent-steering pilot on Can, paired over 20 evaluation seeds. BC succeeds in 12/20 (60%) at every evaluation. "Steered only" and "BC only" count episodes solved by one policy and not the other.

Run	Setting	Online steps	Steered success	Steered only	BC only
C	offline, $\beta = 3$	0	12/20 (60%)	1	1
D	online, $\beta = 3$	2,500	12/20 (60%)	0	0
		5,000	**15/20 (75%)**	3	0
E	online, $\beta = 1$	2,500	14/20 (70%)	2	0
		5,000	11/20 (55%)	2	3
E	online, $\beta = 10$	2,500	13/20 (65%)	1	0
		5,000	13/20 (65%)	1	0

With $\beta = 3$, success rises from 60% to 75% after 5,000 online steps (Table 4, Figure 1a). Three episodes were solved only by the steered policy and none only by BC. A smaller penalty ($\beta = 1$) moves further from BC and ends below it, losing three episodes BC solved. A larger penalty ($\beta = 10$) barely changes behavior (final latent KL 4×10^{-4}). Offline training alone left success unchanged. This ordering fits the role of β, but these results are not conclusive. The 95% Wilson interval for 15/20 is [0.53, 0.89], which overlaps BC's [0.39, 0.78], and only one seed was run.

The action-critic loss (Figure 1b) shows isolated single-batch spikes of up to about 50 against a late-training median of 0.7–0.8. After each spike, the loss returns to its previous level at the next logged update. Its median grows as the TD target grows in magnitude (mean -17 offline, about -64 online), which matches the longer returns of online data. Latent standard deviations stay between 0.96 and 1.0, so the steering policy shows no sign of collapse. GPU updates take 23–28 ms, compared with a median of 41 ms on the CPU nodes. At that rate, we estimated 24–28 hours per full-budget task. The runs in Section 5.4 took 13.6–19.6 hours.

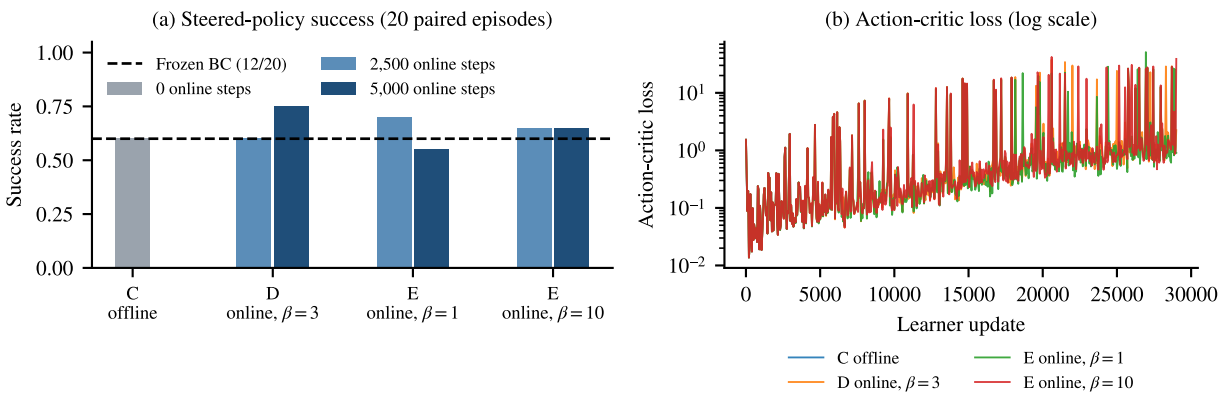


Figure 1: Latent-steering pilot on Can. (a) Steered-policy success at each evaluation; the dashed line is frozen BC on the same seeds. (b) Action-critic loss, logged every 50 updates.

5.4 Full-budget latent steering on Lift, Can, and Square

After the pilot, we ran the full latent-steering budget of Section 4 on all three tasks with $\beta = 3$ and one training seed: 100 BC warmup episodes, then 10^5 online steps at 20 updates per step (2,001,500 updates). The three processes ran concurrently, Lift on one GPU and Can and Square sharing the other, and finished in 13.6, 17.6, and 19.6 hours, with mean update times of 22–28 ms. The frozen flow weights were unchanged at the end of every run. Because these flows were retrained on the GPU workstation, BC was re-measured on the same 50 evaluation seeds (40000–40049) used for the steered policy. It succeeds in 23/50 episodes on Lift, 26/50 on Can, and 4/50 on Square (Table 5), within the intervals of Table 3.

Table 5: Full-budget latent steering, one training seed, $\beta = 3$, 50 paired evaluation episodes every 10^4 online steps. Intervals are 95% Wilson intervals. "Steered only" and "BC only" count episodes at the final evaluation solved by one policy and not the other.

Task	Warmup wins	BC	Steered, 10^4 steps	Steered, final	Best	Steered only	BC only
Lift	51/100	23/50 [0.33, 0.60]	43/50	**46/50** [0.81, 0.97]	48/50	23	0
Can	61/100	26/50 [0.39, 0.65]	39/50	**44/50** [0.76, 0.94]	44/50	18	0
Square	9/100	4/50 [0.03, 0.19]	5/50	0/50 [0.00, 0.07]	5/50	0	4

On Lift and Can, latent steering improves substantially on BC. Steered success exceeds BC at all ten evaluations on both tasks, and the final 95% intervals do not overlap BC's. Most of the gain appears within the first 10^4 online steps (86% on Lift, 78% on Can), followed by smaller changes (Figure 2a). At the final evaluation, the steered policy solves 23 Lift and 18 Can episodes that BC fails, and loses none that BC solves. It also finishes faster: successful episodes average 42 instead of 55 steps on Lift and 111 instead of 119 on Can. Online training episodes show the same trend, with 90 of the last 100 successful on Lift and 94 on Can. These are still single-seed results, so variation across training seeds remains unmeasured.

Square fails. With only 9 successful warmup episodes among about 47,000 transitions, the steered policy stays near BC (0–5/50, against BC's 4/50) for the first 60,000 online steps, then solves no evaluation episode from 70,000 steps onward. The learner logs indicate critic divergence rather than slow learning. Each step earns reward -1, or 4 on success, which ends the episode, so every true value lies in $[-100, 4]$. Square's mean TD target leaves that range at 1.28 million updates (about 64,000 online steps), falls below -300 by 71,000 steps, and ends near -10^3 (Figure 2b). Its late-training action-critic loss has a median near 10^4, compared with 2.7 on Can. The steering policy follows the diverged critic away from the prior. Its median late KL is 2.4, compared with 0.014 on Can and 0.10 on Lift, its latent means reach the ± 2 bound, and its smallest latent standard deviation falls to 0.007. Lift shows a milder version of the same bias: its mean TD target settles near -125, below the reachable minimum, without harming success. Sparse successes combined with 20 updates per step appear to make the critic unstable. Bounding

TD targets to the reachable range, lowering the update ratio, and adding demonstrations or more warmup episodes to replay are the natural fixes.

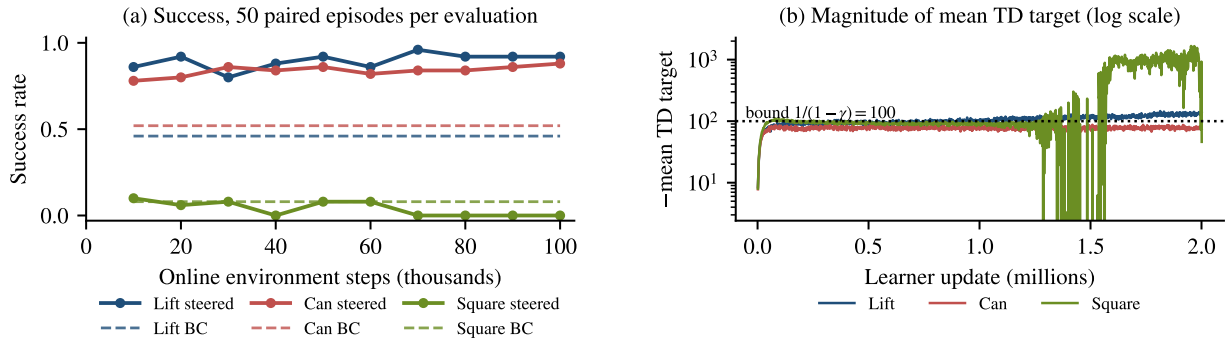


Figure 2: Full-budget latent steering, one training seed. (a) Steered (solid) and frozen BC (dashed) success at each evaluation, against online environment steps after warmup. (b) Magnitude of the mean TD target, logged every 100 updates; the dotted line marks the largest magnitude any true value can reach.

6 Remaining plan

The remaining study will connect value diagnostics to closed-loop learning. We will complete held-out score calibration and a bounded B/C pilot, then run matched online comparisons and add the ungated condition. A small sweep over α_Q will assess sensitivity to score scale. Full-budget latent steering improved on BC for Lift and Can with one training seed (Section 5.4). We will add at least two further seeds for both tasks. For Square, we will bound TD targets to the reachable return range, lower the update-to-data ratio, and seed replay with demonstrations before rerunning. The same GPU setup will host the B/C runs. The current BC configuration retains the final iterate; validation-based checkpoint selection will be examined before fixing the policies and development settings for final evaluation, especially for Lift, where validation loss suggests overfitting. Table 6 lists the remaining work packages and their responsibility fields.

Table 6: Remaining work and responsibility assignments.

Work package	Deliverable	Owner
BC and data	Fixed checkpoints, split verification, and checkpoint selection	*[Name]*
Value estimation	Held-out return calibration, ranking, and density-cost study	*[Name]*
Online comparisons	Matched gating runs and ungated RL condition	*[Name]*
Latent steering	Additional seeds, Square stabilization, and β sensitivity	*[Name]*
Evaluation and writing	Seed statistics, figures, failure analysis, and final report	*[Name]*

The main risk is that BC density may be numerically accurate yet poorly calibrated to task return, making the frozen score difficult to compare with an evolving RL critic. If this prevents useful gating, the fallback is a focused study on one or two tasks that identifies where calibration and ranking break down, using the shared-critic gate as the control. Latent steering provides a second online-improvement result on the same frozen policies whether or not gating succeeds. If density queries are too expensive, we will test Hutchinson estimation or a conditional density surrogate fitted to frozen-flow samples; the surrogate's value calibration will still be measured. Lift and Can take priority if BC behavior or computation makes Square impractical within the course schedule.

References

[1] L. Dodeja et al. *When Life Gives You BC, Make Q-functions: Extracting Q-values from Behavior Cloning for On-Robot Reinforcement Learning*. RSS, 2026. arXiv:2605.05172v3.

[2] H. Hu, S. Mirchandani, and D. Sadigh. *Imitation Bootstrapped Reinforcement Learning*. RSS, 2024. arXiv:2311.02198.

[3] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le. *Flow Matching for Generative Modeling*. 2022. arXiv:2210.02747.

[4] W. Grathwohl, R. T. Q. Chen, J. Bettencourt, I. Sutskever, and D. Duvenaud. *FFJORD: Free-form Continuous Dynamics for Scalable Reversible Generative Models*. 2018. arXiv:1810.01367.

[5] T. Zhang, C. Yu, S. Su, and Y. Wang. *ReinFlow: Fine-tuning Flow Matching Policy with Online Reinforcement Learning*. NeurIPS, 2025. arXiv:2505.22094.

[6] K. Shi et al. *FlowDPG: Deterministic Policy Gradient on Flow Matching Policies for Real-World Manipulation*. 2026. arXiv:2606.22303v2.

[7] Z. Wang et al. *Q-VGM: Q-Guided Value-Gradient Matching for Offline-to-Online RL of Flow-Matching VLA Policies*. 2026. arXiv:2606.08015v5.

[8] A. Wagenmaker et al. *Steering Your Diffusion Policy with Latent Space Reinforcement Learning*. 2025. arXiv:2506.15799v2.

[9] A. Mandlekar et al. *What Matters in Learning from Offline Human Demonstrations for Robot Manipulation*. CoRL, 2021. arXiv:2108.03298.